

AI Repository Architect

Automated Repository Documentation Report

Repository ID: 60f2fed3-b706-46e6-a745-515850a33618

Generated using AI-powered code analysis

Project Overview

This repository contains code for a fantasy RPG duel game that uses a locally-hosted open-source Language Model (LLM) to generate dynamic narrations and visual effects for each chess move. The game simulates a battle between two rival kingdoms, where each chess move corresponds to an action in the battle. The LLM processes move information and returns a dramatic sentence and a matching visual spell effect. If the LLM is unavailable or returns invalid data, the system falls back to predefined narrations and effects to ensure the game continues smoothly.

Architecture

The architecture of this project revolves around a central component: the Language Model (LLM) hosted locally using Ollama. The application interacts with the LLM via HTTP POST requests to generate narrations and effects for chess moves. The system includes fallback mechanisms to handle cases where the LLM is unreachable or returns unparsable data.

Key Components

1. **Language Model (LLM)**: Hosted locally using Ollama, responsible for generating narrations and effects.
2. **Prompt Builder**: Constructs the input prompt for the LLM based on the chess move data.
3. **Fallback Mechanisms**: Provides default narrations and effects when the LLM fails.
4. **Data Handling**: Parses the response from the LLM and handles any exceptions or errors.

Folder Structure

The repository is structured as follows:

...

repository/



```
■■■ main.py # Main script containing the core functionality
■■■ utils.py # Utility functions for data handling and fallbacks
'''
```

Key Components

- **main.py**: Contains the primary logic for interacting with the LLM, including sending requests, parsing responses, and handling fallbacks.
- **utils.py**: Houses utility functions such as building prompts, determining categories of moves, and selecting fallback effects.

Main Functions

- **get_narration(move_info, timeout=8)**: This function sends the move information to the LLM, retrieves the response, and parses it to extract the narration and effect. It also handles exceptions and uses fallbacks if necessary.
- **build_prompt(move_info)**: Constructs a prompt string formatted as JSON, which is sent to the LLM. The prompt includes details about the move and instructions for the LLM to generate a narration and effect.
- **_fallback_narration(move_info)**: Generates a default narration based on the type of move (e.g., checkmate, capture, check, castle, stalemate, normal).
- **_category(move_info)**: Determines the category of the move (e.g., checkmate, capture, check, castle, normal) based on the move information.
- **_fallback_effect(move_info)**: Selects a random visual effect from a predefined list of fallback effects corresponding to the move's category.

API Endpoints

The application does not expose any public API endpoints. Instead, it makes internal HTTP POST requests to the locally-hosted LLM at ``OLLAMA_URL``.

Data Flow

1. **Chess Move Input**: The application receives information about a chess move.
2. **Prompt Construction**: The ``build_prompt`` function constructs a JSON-formatted prompt based on the move information.
3. **LLM Interaction**: The ``get_narration`` function sends the prompt to the LLM via an HTTP POST request.
4. **Response Parsing**: The response from the LLM is parsed to extract the narration and effect.
5. **Error Handling**: If the LLM is unreachable or returns invalid data, the application uses fallback mechanisms to provide default narrations and effects.
6. **Output**: The final narration and effect are returned and can be used by the frontend to animate the battle scene.

Setup Instructions

1. **Install Dependencies**:

- Ensure Python is installed on your system.
- Install the required Python packages using pip:

```
```bash
pip install requests
```
```

2. **Set Up Ollama**:

- Download and install Ollama from ollama.com.
- Start the Ollama service and ensure it is running locally.
- Set the ``OLLAMA_URL`` environment variable to point to the URL where your Ollama service is hosted (typically ``http://localhost:11434``).

3. **Configure Model Name**:

- Set the `MODEL\_NAME` environment variable to the name of the model you want to use (e.g., `llama3`, `mistral`, `qwen`).

4. **Run the Application**:

- Execute the main script:

```
```bash
```

```
python main.py
```

```
```
```

Future Improvements

1. **Enhanced Error Handling**: Improve error handling to provide more detailed feedback about issues with the LLM interaction.
2. **Customizable Fallbacks**: Allow customization of fallback narrations and effects through configuration files.
3. **User Interface**: Develop a user-friendly interface for